```
[v1 → null, v2 → v3, v3 → v1, v4 → v3, v5 → v3, v6 → v5].
```

If we start at v_5, for instance, we see that `parents[`v_5`]==` v_3, and this means that we need to add v_3 to `path` after v_5, and then look at its parent, etc...

Right before line #7, we have `path==[`v_5`,` v_3`,` v_1`]`, and reversing it we get the sequence of vertices we were looking for.

14.3.4 Depth First Search

BFS, as we have seen, uses a clear strategy to traverse a graph: it starts from the closest vertices to a source, and the it propagates in all directions like a wave, in concentric rings, first all the vertices at distance 1, then the next ring, with all the vertices at distance 2, etc...

This strategy is, as we have seen, quite effective when we have to find the shortest paths from a source to the other vertices in a graph; at the same time, of course, this is not the only possible strategy to traverse a graph.

A different approach, for instance, is to traverse paths “in depth”: it’s like when you are in a maze, looking for the exit: you keep going as far away from the source as possible, choosing your direction at each intersection, until you hit a dead end (in graph terms, until you get to a vertex without any untraveled outgoing edges). At that point, you retrace your steps up to the previous bifurcation (the previous vertex